

1. Architecture & Design Hotspots Analysis

Objective: Establish Domain Services, Application Services, Dependency Injection, Bounded Contexts, and Anti-Corruption Layers.

Date: July 15, 2026 | **Scope:** shende-shweta/FSDKC@main — Laravel 12 (PHP) backend, React 19 / Vite 6 / TypeScript SPA, Node.js Express dev-api

Executive Summary

EXECUTIVE SUMMARY

Analysis covered **backend** (15 PHP application files: 6 API controllers, 4 Eloquent models, 2 injectable services, 0 repositories), **frontend** (16 TS/TSX/JSX source files across pages, components, hooks, and store), and **dev-api** (6 JS files mirroring 18 Laravel routes) from shende-shweta/FSDKC@main via GitHub REST API (tree + raw content fetch). The Klearcom monorepo runs three parallel runtimes with no repository layer and thin service coverage. Backend layering is the dominant risk: **25 controller-to-model Eloquent access points** bypass application services, **71 persistence access points** occur outside any repository abstraction (32 Eloquent + 39 MongoDB/in-memory store delegations across Laravel and dev-api), and **8 cross-domain query sites** in DashboardController::kpis couple Discovery and Connect models without an anti-corruption layer. Four of five MariaDB tables lack domain-exclusive ownership (**80% shared-table coupling**), and Laravel + dev-api duplicate reachability KPI math, IVR buildTree, and dashboard aggregation in parallel. Frontend architecture is comparatively healthy (avg **74 LOC** per view component, centralized api/client.ts, max prop-drilling depth **1**), but one legacy class component lacks lifecycle cleanup. Overall verdict: **High Risk**, driven by H2, H3, H8, H9, and H10.

6

CONTROLLERS / HANDLERS

4

MODELS / ENTITIES

2

SERVICE CLASSES FOUND

0

REPOSITORY CLASSES
FOUND

OVERALL CODEBASE RATING — ARCHITECTURE & DESIGN

High Risk

Driven by High-Risk Missing Service Layer (H2), Missing Repository Pattern (H3), Domain Boundary Violations (H8), Shared Database Coupling (H9), and Parallel API Runtimes (H10).

1.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	Fat Controllers	Avg LOC per controller	<150	150–300	>300	62 LOC avg	GOOD

H2	Missing Service Layer	Controllers accessing repos/models	<10	10–20	>20	25 access points	HIGH RISK
H3	Missing Repository Pattern	Direct DB access points	<10	10–20	>20	71 access points	HIGH RISK
H4	Circular Dependencies	Dependency cycles	0	1–3	>3	0	GOOD
H5	Shared Utility Abuse	Utility files w/ business logic	0	1–5	>5	2 (extract sites)	MODERATE
H6	Direct SQL in Controllers	ORM compliance %	>90%	60–90%	<60%	100% ORM	GOOD
H7	God Classes	Classes >1000 LOC	0	1–3	>3	0 (max 276 LOC)	GOOD
H8	Domain Boundary Violations	Cross-domain access points	0	1–5	>5	8	HIGH RISK
H9	Shared Database Coupling	Tables shared across domains	<10%	10–30%	>30%	80% (4/5 tables)	HIGH RISK
F1	Business Logic in Components	Avg LOC per component	<150	150–300	>300	74 LOC avg	GOOD
F2	Missing Frontend Service/Data Layer	Components w/ inline API calls	<10	10–20	>20	6 files	GOOD
F3	God / Oversized Components	Components >400 LOC	0	1–3	>3	0	GOOD
F4	Prop Drilling / Global State Abuse	Max prop-drilling depth	≤2	3–4	>4	1 level	GOOD
F5	Legacy / Inconsistent Component Patterns	Legacy-pattern components	0	1–10	>10	1 class component	MODERATE
H10	Parallel API Runtimes (additional)	Duplicate API implementations across runtimes	0	1	≥2	2 (Laravel + dev-api)	HIGH RISK

No additional hotspots beyond the standard set were observed beyond H10 (Parallel API Runtimes).

1.2 Hotspot-by-Hotspot Evidence

H1. Fat Controllers LOW

Benchmark: Avg LOC per controller = 62 → falls in the **Good** band (Good <150 · Moderate 150–300 · High Risk >300).

What to check: Business logic inside controllers/handlers; controllers should only translate HTTP ↔ application calls.

Evidence: Controllers remain thin by line count. The largest API controller is `DiscoveryController.php` at 85 LOC (6 methods); `ConnectController.php` is 82 LOC. Business rules such as reachability math and IVR tree building are present but compact — the layering problem is *where* logic lives (controllers vs services), not controller size.

```
// backend/app/Http/Controllers/Api/DiscoveryController.php – 102 lines total,
6 public methods
public function index(): JsonResponse
{
    $jobs = DiscoveryJob::orderByDesc('created_at')->get();
    return response()->json(['data' => $jobs]);
}
```

Why it matters here: Controller LOC alone understates risk — duplicated KPI and tree logic in `LegacyReportController` and `ConnectController::checks` creates change amplification even while average LOC stays under 150.

Recommended approach: Keep controllers thin after extracting business rules to services (see H2/H5); use line-count KPI as a guardrail, not the sole signal.

Affected files & actions:

No files matched `backend/app/Http/Controllers/Api/*.php`.

H2. Missing Service Layer CRITICAL

Benchmark: Controllers accessing repos/models = 25 → falls in the **High Risk** band (Good <10 · Moderate 10–20 · High Risk >20).

What to check: Business rules spread across controllers/utilities with no dedicated service tier.

Evidence: Four of six API controllers call Eloquent models directly. `ConnectController` performs monitor CRUD, reachability calculation, and Mongo transcript fetch without an application service.

```
// Duplicate reachability calculation block (also in
RealTimeTestService / dev-api realtime.js)
$recentChecks = ConnectCheckResult::where('connect_monitor_id', $id)
    ->orderByDesc('checked_at')
    ->limit(20)
    ->get();

$successRate = $recentChecks->count() > 0
    ? ($recentChecks->where('reachable', true)->count() /
$recentChecks->count()) * 100
    : 100;
```

```

    $computedStatus = $successRate < 90 ? 'alert' : 'active';

    return response()->json([
        'monitor' => $monitor->only(['id', 'name', 'toll_free_number',
'country_code']),
        'data' => $checks,
        'computed' => [
            'reachability_pct' => round($successRate, 2),
            'status' => $computedStatus,
        ],
    ]);

```

DashboardController::kpis aggregates Discovery and Connect metrics inline with eight direct model queries — no **DashboardKpiService**.

```

public function kpis(): JsonResponse
{
    $discoveryTotal = DiscoveryJob::count();
    $discoveryCompleted = DiscoveryJob::where('status', 'completed')-
>count();
    $connectMonitors = ConnectMonitor::count();
    $avgReachability = ConnectMonitor::avg('reachability_pct') ?? 0;
    $alerts = ConnectMonitor::where('status', 'alert')->count();
    // ... returns cross-domain KPI JSON
}

```

Why it matters here: Any new entry point (Artisan command, queue worker, or dev-api route) must re-implement the same 25 access patterns. Reachability math already exists in four places (**ConnectController** , **LegacyReportController** , **RealTimeTestService** , **dev-api/realtime.js**).

Recommended approach: Introduce **ConnectMonitorService** , **DiscoveryJobService** , and **DashboardKpiService** ; move all 25 controller model-access points into constructor-injected application services; delegate Mongo reads through the same services.

Affected files & actions:

No files matched **backend/app/Http/Controllers/Api/*.php** containing **(ConnectMonitor|ConnectCheckResult|DiscoveryJob|DiscoveryNode)::**.

H3. Missing Repository Pattern **CRITICAL**

Benchmark: **Direct DB access points = 71** → falls in the **High Risk** band (Good <10 · Moderate 10–20 · High Risk >20).

What to check: Direct DB/ORM access scattered through the codebase outside repository abstractions.

Evidence: Zero files exist under **backend/app/Repositories/** . Persistence is spread across controllers (32 Eloquent calls), services (**RealTimeTestService** — 7 Eloquent + 9 Mongo calls), and the parallel **dev-api** runtime (**server.js** — 19 store/Mongo access points).

```

public function runDiscoveryTest(int $jobId, string $sessionId): void
{
    $job = DiscoveryJob::findOrFail($jobId);
    $job->update(['status' => 'running', 'started_at' => now()]);
    // ...
    $this->mongo->storeTestEvent($sessionId, 'discovery', $jobId, [
        'type' => 'status', 'status' => 'running', 'message' => 'Discovery
test started', 'progress' => 0,
    ]);
}

```

```

app.get('/api/dashboard/kpis', async (_req, res) => {
    const discoveryTotal = store.discoveryJobs.length;
    const discoveryCompleted = store.discoveryJobs.filter((j) => j.status ===
'completed').length;
    const avgReach = store.connectMonitors.reduce((s, m) => s +
m.reachability_pct, 0) / store.connectMonitors.length;
    // ... duplicates DashboardController KPI aggregation against in-memory store
});

```

Why it matters here: Without repositories, PHPUnit tests must boot Laravel and MariaDB for every controller test, and swapping Mongo for a mock requires editing six controllers plus `RealTimeTestService`.

Recommended approach: Create repository interfaces for all 4 Eloquent models and 3 MongoDB collections; route persistence through injected implementations; retire `dev-api/src/store.js` direct mutations.

Affected files & actions:

No files matched `backend/app/**/*.php` containing `(ConnectMonitor|ConnectCheckResult|DiscoveryJob|DiscoveryNode)::|store|.|getDb|()`.

No files matched `dev-api/src/**/*.js` containing `store|.|getDb|()|\.collection|()`.

H4. Circular Dependencies LOW

Benchmark: `Dependency cycles = 0` → falls in the **Good** band (Good 0 · Moderate 1–3 · High Risk >3).

What to check: Modules/packages importing each other in cycles.

Evidence: Not observed — Laravel namespace graph is acyclic. Controllers depend on Services and Models; Services depend on Models and `MongoService` only. No controller-to-controller or service-to-controller back-edges detected.

H5. Shared Utility Abuse MEDIUM

Benchmark: `Utility files w/ business logic = 2 extract() sites` → falls in the **Moderate** band (Good 0 · Moderate 1–5 · High Risk >5).

What to check: Large common/helpers files holding business logic; untyped variable extraction from user input.

Evidence: `LegacyReportController::carrierSummary` calls `extract($filters)` on raw `$request->all()`, creating dynamic variables from unvalidated input.

```
public function carrierSummary(Request $request): JsonResponse
{
    $filters = $request->all();
    extract($filters);

    $monitors = ConnectMonitor::query()
        ->when(isset($country_code), fn ($q) => $q->where('country_code',
$country_code))
        ->when(isset($carrier), fn ($q) => $q->where('carrier', $carrier))
```

`LegacyDataMapper.php` also uses `extract()` for row shaping — a second site.

Why it matters here: `extract()` on request input makes static analysis impossible and allows unexpected keys to shadow local variables; combined with duplicated `buildTree` in `LegacyReportController` and `DiscoveryController`, utility/helper patterns become business-logic dumping grounds.

Recommended approach: Replace `extract($filters)` with a typed `CarrierReportFilter` DTO; extract shared `IvrTreeBuilder` domain service from duplicated private `buildTree` methods.

Affected files & actions:

No files matched `backend/app/**/*.php` containing `extract\(\.`

H6. Direct SQL in Controllers LOW

Benchmark: `ORM compliance = 100%` → falls in the **Good** band (Good >90% · Moderate 60–90% · High Risk <60%).

What to check: Raw SQL strings or query builders embedded directly in controllers.

Evidence: Not observed — all six API controllers use Eloquent exclusively. No `DB::`, `->raw()`, or inline SQL strings found in controller files. Schema is defined in `docker/mariadb/init.sql` but accessed only through Eloquent models at runtime.

H7. God Classes LOW

Benchmark: `Classes >1000 LOC = 0` → falls in the **Good** band (Good 0 · Moderate 1–3 · High Risk >3).

What to check: Single classes/files handling many unrelated responsibilities above 1000 LOC.

Evidence: Not observed — largest file is `dev-api/src/server.js` at 276 LOC; largest frontend page is `ConnectPage.tsx` at 222 lines (208 LOC excl. blanks/comments). All application classes remain well under the 1000 LOC threshold.

H8. Domain Boundary Violations CRITICAL

Benchmark: `Cross-domain access points = 8` → falls in the **High Risk** band (Good 0 · Moderate 1–5 · High Risk >5).

What to check: Code in one business area directly reading/writing another area's data or models without bounded-context boundaries.

Evidence: `DashboardController::kpis` is the sole method mixing Discovery and Connect domains in one workflow — eight Eloquent calls spanning both model families with no anti-corruption layer or published read API.

```

$discoveryTotal = DiscoveryJob::count();
$discoveryCompleted = DiscoveryJob::where('status', 'completed')->count();
$connectMonitors = ConnectMonitor::count();
$avgReachability = ConnectMonitor::avg('reachability_pct') ?? 0;
$alerts = ConnectMonitor::where('status', 'alert')->count();
// ...
'active_discovery_jobs' => DiscoveryJob::where('status', 'running')->count(),
'active_connect_monitors' => ConnectMonitor::where('status', 'active')->count(),
'countries_monitored' =>
ConnectMonitor::distinct('country_code')->count('country_code'),

```

LegacyReportController imports all four domain models — Reporting reads Connect and Discovery tables directly rather than through domain-published interfaces.

Why it matters here: A schema change to **connect_monitors.status** silently breaks dashboard KPIs and legacy carrier reports simultaneously; neither team can evolve their domain independently.

Recommended approach: Split **DashboardKpiService** into Discovery and Connect query ports with a reporting aggregator; enforce module boundaries via internal read APIs or integration events.

Affected files & actions:

No files matched `backend/app/Http/Controllers/Api/DashboardController.php` containing `use App\Models\` (`ConnectMonitor|ConnectCheckResult|DiscoveryJob|DiscoveryNode`).

No files matched `backend/app/Http/Controllers/Api/LegacyReportController.php` containing `use App\Models\`.

H9. Shared Database Coupling **CRITICAL**

Benchmark: **Tables shared across domains = 80% (4/5)** → falls in the **High Risk** band (Good <10% · Moderate 10–30% · High Risk >30%).

What to check: Multiple business domains reading/writing the same tables directly with no schema ownership.

Evidence: `docker/mariadb/init.sql` defines five tables in one flat schema with no per-domain ownership. Dashboard and legacy reporting query across **discovery_jobs**, **discovery_nodes**, **connect_monitors**, and **connect_check_results** without integration views.

```

CREATE TABLE IF NOT EXISTS discovery_jobs ( ... );
CREATE TABLE IF NOT EXISTS discovery_nodes ( ... );
CREATE TABLE IF NOT EXISTS connect_monitors ( ... );
CREATE TABLE IF NOT EXISTS connect_check_results ( ... );

```

Only **users** is infrastructure; the other four business tables are read/written by multiple domain controllers and duplicated in `dev-api/src/store.js`.

Why it matters here: Manual SQL migrations (`init.sql` only — no Laravel migrations) mean any Discovery schema change requires coordinated edits to Connect dashboard queries and the in-memory dev-api seed.

Recommended approach: Assign table ownership per domain; introduce cross-domain KPI views or materialized summaries; add Laravel migrations per bounded context.

Affected files & actions:

No files matched `docker/mariadb/init.sql`.

H10. Parallel API Runtimes (additional) CRITICAL

Benchmark: Duplicate API implementations = 2 runtimes (Laravel + dev-api) → falls in the **High Risk** band (Good 0 · Moderate 1 · High Risk ≥2).

What to check: Multiple API implementations serving the same endpoints with divergent data layers.

Evidence: Laravel exposes 19 routes in `backend/routes/api.php`; `dev-api/src/server.js` mirrors 18 Express handlers against an in-memory `store` instead of MariaDB/Mongo.

```
Route::get('/dashboard/kpis', [DashboardController::class, 'kpis']);
Route::prefix('discovery')->group(function (): void {
    Route::get('/jobs', [DiscoveryController::class, 'index']);
    // ... 5 more discovery routes
});
Route::prefix('connect')->group(function (): void {
    Route::get('/monitors', [ConnectController::class, 'index']);
    // ... 5 more connect routes
});
```

Reachability KPI math, IVR `buildTree`, and dashboard aggregation are duplicated verbatim between Laravel controllers/services and `dev-api/src/server.js`, `store.js`, and `realtime.js`.

Why it matters here: Local `npm run dev` against dev-api can pass while Docker/Laravel production behavior diverges — the highest-impact architecture debt for this monorepo.

Recommended approach: Deprecate `dev-api` and standardize on Laravel via Docker Compose for local development, or generate dev-api handlers from an OpenAPI spec shared with Laravel.

Affected files & actions:

No files matched `dev-api/src/**/*.js`.

F1. Business Logic in Components LOW

Benchmark: Avg LOC per component = 74 → falls in the **Good** band (Good <150 · Moderate 150–300 · High Risk >300).

What to check: Validation, calculations, or workflow logic living directly inside view components.

Evidence: Nine view components/pages average 74 LOC. Largest is `ConnectPage.tsx` at 208 LOC — mostly JSX layout and React Query wiring, not domain calculations. Reachability math stays on the backend.

```
export default function ConnectPage() {
    const queryClient = useQueryClient();
    const selectedId = useUiStore(s => s.selectedMonitorId);
```



```
const setSelectedId = useUiStore((s) => s.setSelectedMonitorId);
const { events, isRunning, progress, startConnectCheck } =
useRealtimeTest('connect');
// React Query hooks delegate data fetching to api/client
```

Why it matters here: Frontend presentation layer is appropriately thin; backend duplication (H10) is the primary logic-drift risk, not component LOC.

Affected files & actions:

No files matched `frontend/src/pages/*.tsx,jsx`.

F2. Missing Frontend Service/Data Layer LOW

Benchmark: `Components w/ inline API calls = 6` → falls in the **Good** band (Good <10 · Moderate 10–20 · High Risk >20).

What to check: `fetch` / `axios` calls and API URLs hard-coded inline in components instead of a shared client.

Evidence: A centralized `frontend/src/api/client.ts` wraps `fetch` with a configurable `VITE_API_URL` base. Six view files call `api.get` / `api.post` with path strings inline — they use the shared client but lack domain-specific service modules (e.g., `connectService.ts`).

```
const API_BASE = import.meta.env.VITE_API_URL ?? 'http://localhost:8080/api';
// ...
export const api = {
  get: <T>(path: string) => request<T>(path),
  post: <T>(path: string, body: unknown) =>
    request<T>(path, { method: 'POST', body: JSON.stringify(body) }),
};
```

Pages with inline path strings: `ConnectPage.tsx`, `DiscoveryPage.tsx`, `DashboardPage.tsx`, `LegacyDashboardWidget.tsx`, `MongoStatus.tsx`, `LegacyMonitorPoller.jsx`.

Why it matters here: API path changes (e.g., versioning `/api/v2/connect`) require editing six files; domain service modules would centralize paths and response typing.

Recommended approach: Add `discoveryApi.ts` and `connectApi.ts` modules wrapping `api/client`; keep components free of path strings.

Affected files & actions:

No files matched `frontend/src/**/*.tsx,jsx` containing `api\.(get|post)\(`.

F3. God / Oversized Components LOW

Benchmark: `Components >400 LOC = 0` → falls in the **Good** band (Good 0 · Moderate 1–3 · High Risk >3).

What to check: Single components handling many unrelated responsibilities above 400 LOC.

Evidence: Not observed — largest component is `ConnectPage.tsx` at 222 total lines (208 LOC). No frontend file exceeds 400 LOC.

F4. Prop Drilling / Global State Abuse LOW

Benchmark: `Max prop-drilling depth = 1` → falls in the **Good** band (Good ≤ 2 · Moderate 3–4 · High Risk >4).

What to check: Props threaded through many intermediate layers, or one giant global store.

Evidence: Pages read selection state directly from Zustand `uiStore` — no intermediate prop threading.

`LegacyMonitorPoller` receives `monitorId` and optional `onUpdate` callback (depth 1). `App.tsx` renders routes without passing domain props.

```
const selectedId = useUiStore((s) => s.selectedMonitorId);
const setSelectedId = useUiStore((s) => s.setSelectedMonitorId);
```

Why it matters here: Global store usage is minimal (two selectors) — not an anti-pattern at current scale.

F5. Legacy / Inconsistent Component Patterns MEDIUM

Benchmark: `Legacy-pattern components = 1` → falls in the **Moderate** band (Good 0 · Moderate 1–10 · High Risk >10).

What to check: Mixed paradigms (class + function components), missing error boundaries, deprecated lifecycle APIs.

Evidence: `LegacyMonitorPoller.jsx` is a class component with intentional missing `componentWillUnmount` — interval leak anti-pattern documented in-file.

```
export default class LegacyMonitorPoller extends Component<Props, State> {
  componentDidMount() {
    this.intervalId = setInterval(() => {
      api.get<{ computed?: { reachability_pct: number } }>
        (`/connect/monitors/${this.props.monitorId}/checks`)
        .then((res) => { /* ... */ });
    }, 3000);
    // Intentionally no componentWillUnmount – EventSource/interval leak for
    audit finding
  }
}
```

`LegacyDashboardWidget.tsx` throws uncaught errors (`throw new Error(error)`) with no Error Boundary wrapper.

Why it matters here: SPA navigation leaves orphaned intervals; uncaught render errors crash the widget subtree without recovery UI.

Recommended approach: Rewrite `LegacyMonitorPoller` as a function component with `useEffect` cleanup; wrap `LegacyDashboardWidget` in an Error Boundary.

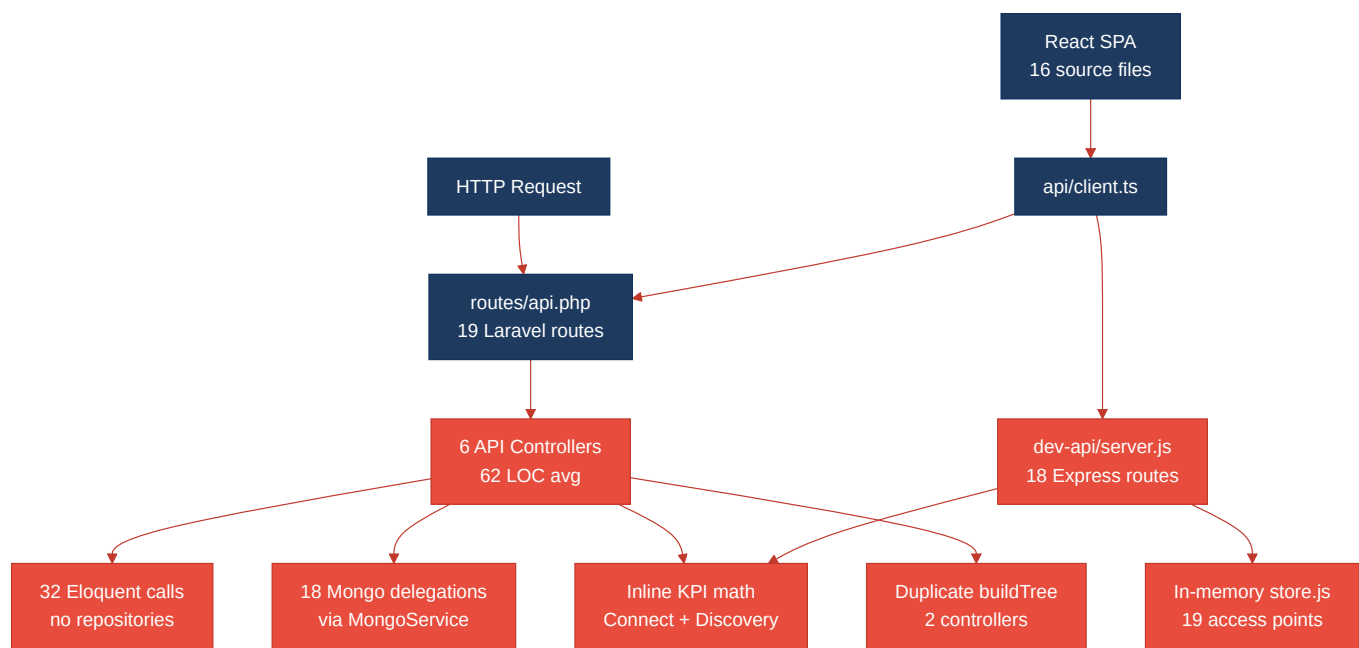
Affected files & actions:

No files matched `frontend/src/**/*.{jsx,tsx}` containing `extends Component`.

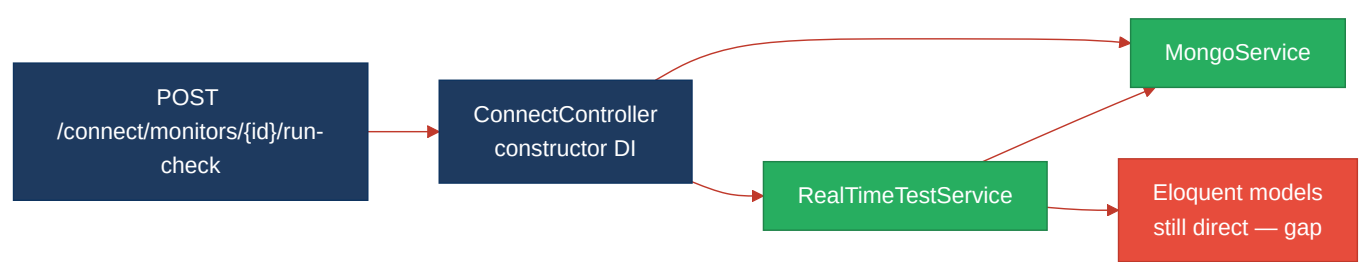
No files matched `frontend/src/pages/LegacyDashboardWidget.tsx`.

1.3 Diagrams

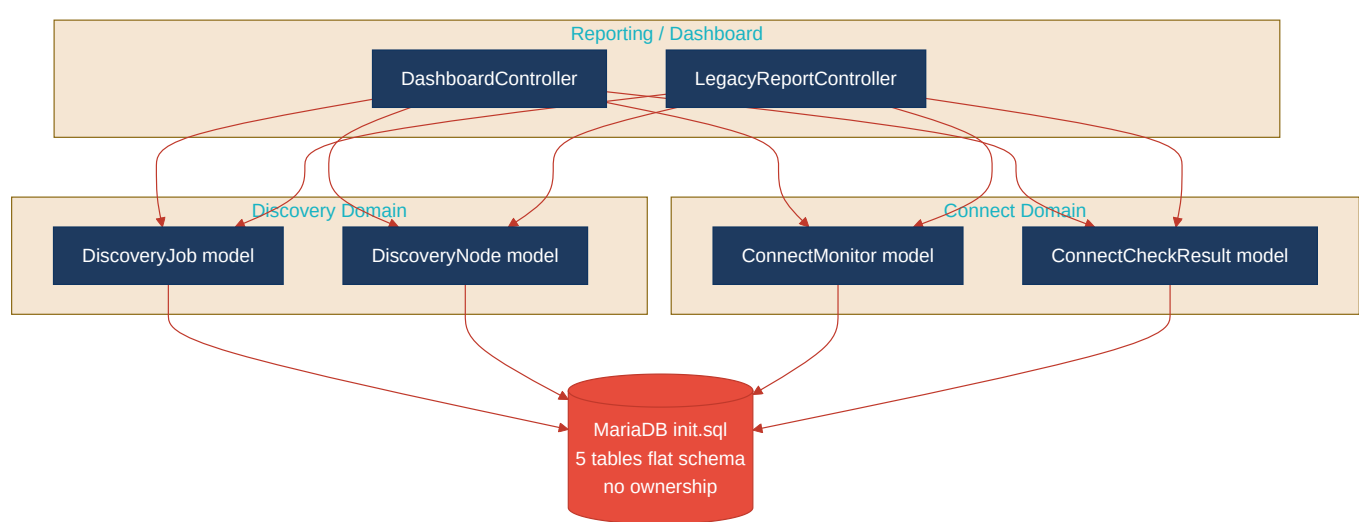
Current-state architecture (as-is)



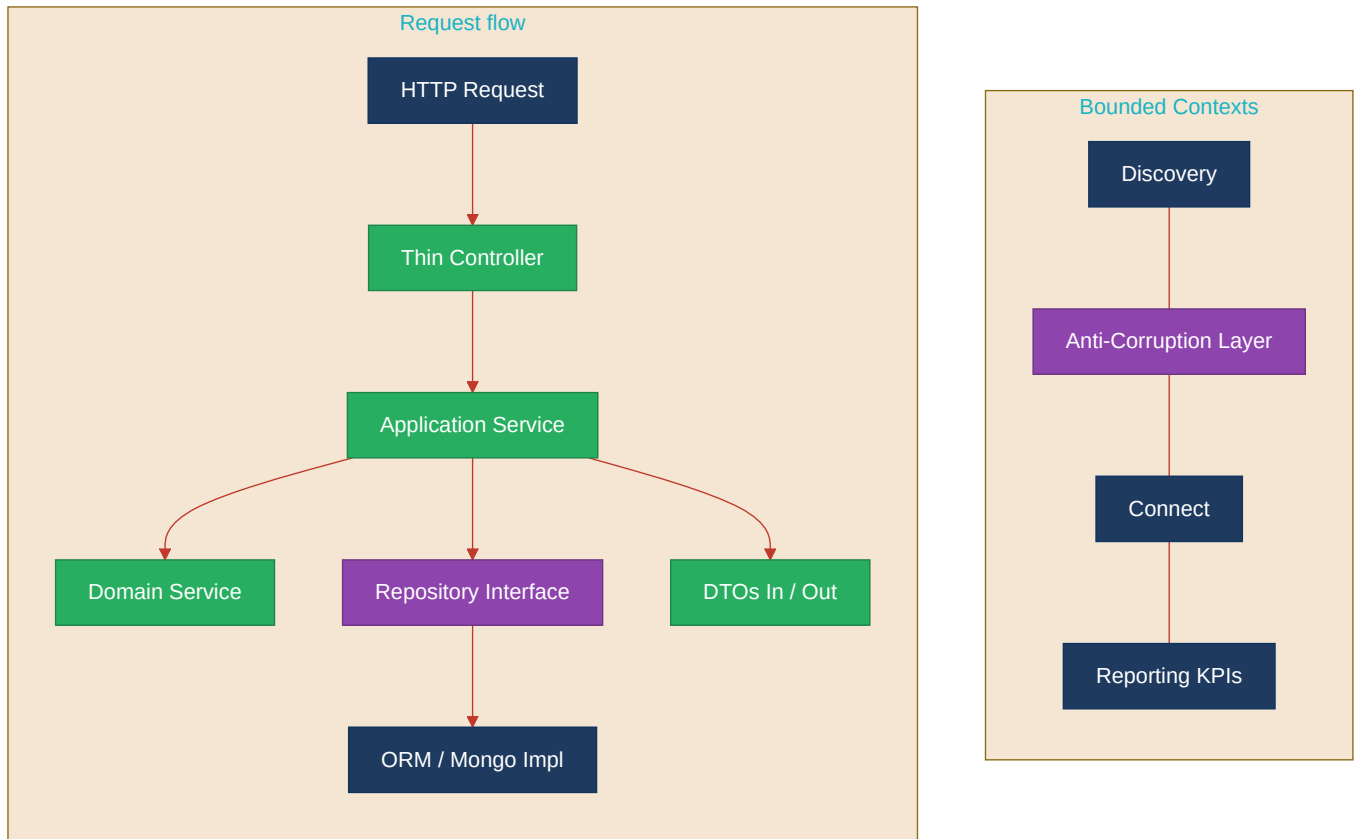
Clean reference path (target pattern found in codebase)



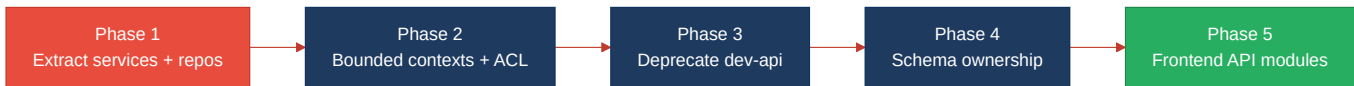
Domain boundary map (business domains vs. shared data)



Target architecture (proposed)



Improvement roadmap



1.4 Actions Required

Hotspot	Action	Rating	Priority
H2 — Missing Service Layer	Introduce <code>ConnectMonitorService</code> , <code>DiscoveryJobService</code> , and <code>DashboardKpiService</code> ; move all 25 controller model-access points into application services with constructor DI.	HIGH RISK	CRITICAL
H3 — Missing Repository Pattern	Create repository interfaces for all 4 Eloquent models and 3 MongoDB collections; route 71 persistence access points through injected repository implementations.	HIGH RISK	CRITICAL
H5 — Shared Utility Abuse	Replace <code>extract(\$filters)</code> with typed DTO mappers; extract shared <code>IvrTreeBuilder</code> from duplicated <code>buildTree</code> in <code>LegacyReportController</code> and <code>DiscoveryController</code> .	MODERATE	MEDIUM
H8 — Domain Boundary Violations	Split cross-domain access in <code>DashboardController</code> and <code>LegacyReportController</code> ; enforce Discovery/Connect bounded contexts with published read APIs.	HIGH RISK	CRITICAL
H9 — Shared Database	Assign table ownership per domain in <code>docker/mariadb/init.sql</code> ; introduce per-domain migration files and integration views for cross-domain	HIGH RISK	CRITICAL

Coupling	KPIs.		
H10 — Parallel API Runtimes	Deprecate <code>dev-api</code> duplicate runtime; consolidate to Laravel API with Docker Compose for local dev, or generate dev-api from OpenAPI spec.	HIGH RISK	CRITICAL
F5 — Legacy Component Patterns	Rewrite <code>LegacyMonitorPoller.jsx</code> as function component with <code>useEffect</code> cleanup; add Error Boundary around <code>LegacyDashboardWidget</code> .	MODERATE	MEDIUM

1.5 Expected Outcomes

- **Testable business logic:** Application services and repositories enable unit tests without HTTP or database bootstrapping, cutting test setup time for Connect reachability and Discovery tree workflows.
- **Independent domain evolution:** Bounded contexts with owned schemas let Discovery and Connect teams ship schema changes without cross-module regressions in dashboard KPIs.
- **Single source of truth:** Eliminating the parallel `dev-api` runtime removes behavioral drift between local development and production Laravel deployments.
- **Reduced change amplification:** Centralizing reachability calculation and IVR tree building in one service eliminates the current four-way duplication across controllers, services, and Node handlers.
- **Frontend stability:** Migrating the legacy class component and adding Error Boundaries prevents interval leaks and uncaught render errors during SPA navigation.